

PetPal Tracker — HTTP API Reference

Technical reference for **tracker-tcp-server** (Express on port **5002**). Collars connect over **TCP**; the web app and Postman call **HTTP**.

PDF: `API_REFERENCE.pdf` — regenerate with `npm run docs:pdf` in `tracker-tcp-server`.

Transport	Port (default)	Env var	Protocol
Xexun TCP	5001	TCP_PORT	FC ... CF binary frames
365GPS TCP	5003	GPS365_TCP_PORT	7878 ... 0D0A frames
HTTP + static UI	5002	HTTP_PORT	REST JSON

Base URL (production example): `http://116.203.209.68:5002`

Discovery endpoints:

- `GET /api` — Xexun command API index
- `GET /api/g365` — 365GPS command API index

Authentication

No API keys are required today. CORS allows browser `GET / POST` from configured origins (`HTTP_CORS_ORIGIN`, default `*`).

Optional audit header (logged only): `X-User` or `X-User-Id`.

App API (`/api/app/*`)

Used by the **PetPal web UI**. Read-only except home pin. Data comes from **SQLite** (`data/petpal.sqlite`).

`GET /api/app/devices`

List all devices last seen on TCP.

Response: JSON array of device snapshots.

`GET /api/app/devices/:imei`

Single device snapshot.

Status	Body
200	Device object

404	{ "error": "not_found" }
-----	--------------------------

GET /api/app/position

Latest position for maps / Live tab.

Query: deviceId or imei (required)

Response (200): Normalized position object, e.g.:

```
{
  "imei": "863235081917526",
  "lat": 34.964747,
  "lng": 33.126694,
  "source": "gps",
  "accuracy": "high",
  "gpsValid": true,
  "battery": 99,
  "signal": 23,
  "receivedAt": "2026-05-30T12:00:04.938Z",
  "lastUpdate": "2026-05-30T12:00:04.938Z",
  "secondsAgo": 42,
  "freshness": "live",
  "speed": 0.6
}
```

Wi-Fi at home (no GPS coords in packet):

```
{
  "imei": "...",
  "lat": 34.96,
  "lng": 33.12,
  "homeLat": 34.96,
  "homeLng": 33.12,
  "atHomeWifi": true,
  "source": "wifi",
  "locationKind": "home_wifi",
  "gpsValid": false,
  "wifiBssids": ["aa:bb:cc:dd:ee:ff"]
}
```

Status	Body
400	{ "error": "missing_deviceId" }
404	{ "error": "not_found" } or { "error": "no_position" }

Rules:

- GPS/LBS coordinates are returned when plausible.
- Wi-Fi packets contain BSSIDs only; if home was saved, lat / lng = home pin.

- receivedAt / lastUpdate = **server receive time** (use for History date filters).

GET /api/app/history

Position history for the History tab.

Query:

Param	Required	Description
deviceId or imei	yes	Collar IMEI
limit	no	Max rows (default 100, max 20000 in range mode)
from	no	ISO 8601 start (server received_at)
to	no	ISO 8601 end (inclusive)

Response (200):

```
{
  "imei": "863235081917526",
  "history": [
    {
      "lat": 34.964747,
      "lng": 33.126694,
      "source": "gps",
      "battery": 99,
      "signal": 23,
      "timestamp": "2026-05-30T21:00:04.938Z",
      "receivedAt": "2026-05-30T21:00:04.938Z",
      "deviceTimeUtc": "2026-05-30T22:52:46.000Z",
      "gpsValid": true,
      "accuracy": "high"
    }
  ],
  "calendarMatch": true,
  "totalInRange": 8272,
  "truncated": false
}
```

Field	Meaning
calendarMatch	false if no rows matched from / to and server returned latest fixes instead
totalInRange	Total rows in DB for the window (when from + to set)
truncated	true when history.length < totalInRange (raise limit or narrow the window)

Points are ordered **oldest** → **newest** by server receive time.

POST /api/app/home

Save **home map pin** for Wi-Fi-at-home display.

Body:

```
{
  "deviceId": "863235081917526",
  "lat": 34.96475,
  "lng": 33.12669
}
```

Status	Body
200	{ "ok": true, "imei": "...", "homeLat": ..., "homeLng": ... }
400	missing_deviceId, missing_lat_lng, invalid_coordinates
501	home_location_not_supported (SQLite disabled)

Legacy HTTP (same data, shorter paths)

Method	Path	Notes
GET	/devices	Same as /api/app/devices
GET	/devices/:imei	Device snapshot
GET	/devices/:imei/status	Battery, signal, moving, charging, steps
GET	/position?deviceId=	Same payload as /api/app/position

Xexun command API

Queues **text commands** sent as Xexun **0x21** frames on the device's **next TCP uplink** (after ACK). Device must be online on **port 5001**.

Path prefixes (equivalent):

- /commands/*
- /api/tracker/commands/*

GET .../pending/:imei

Returns { "imei": "...", "pending": [{"ip=...", "tk=..."}] }.

Command endpoints (all POST, JSON body with imei)

Path	Body	Queued command
/queue	{ imei, command }	Raw string
/ip-transfer	{ imei, host, port? }	ip=host:port

/ip/query	{ imei }	ip=?
/apn	{ imei, apn }	APN=...
/tracking	{ imei, tk } or { imei, p1...p7 }	tk=...
/tracking/query	{ imei }	tk=?
/power-off	{ imei }	of=1
/restart	{ imei }	rt=1
/message	{ imei, text }	mg=...
/timezone	{ imei, tz }	tz=... (-12...+12)
/timezone/query	{ imei }	tz=?
/ble	{ imei, bssid_list } / { clear: true } / { query: true }	ble=...
/wifi	{ imei, bssid_list } / { clear: true } / { query: true }	wifi=...

Success (200):

```
{
  "ok": true,
  "imei": "869469088344608",
  "command": "ip=116.203.209.68:5001",
  "commandId": 42,
  "pending": ["ip=116.203.209.68:5001"],
  "note": "Sent on next TCP uplink (0x20) after ACK."
}
```

Tracking preset example (tk seven fields per vendor PDF):

```
{
  "imei": "869469088344608",
  "tk": "2,60,2,3600,0,0,20"
}
```

Mode 2 = timed GPS; 3 = Wi-Fi priority (indoor).

365GPS command API (/api/g365/*)

Sends binary frames **immediately** on the device's active TCP session on **port 5003**. Device must be **online on 5003** (not 5001).

Index: GET /api/g365

Protocol details: G365_PROTOCOL.md

Method	Path	Body	Description
POST	/api/g365/commands/server-redirect	{ imei, host, port? }	0x66 — device reconnects to new IP:port
POST	/api/g365/commands/manual-position	{ imei, mode?: "gps" "wifi" }	0x80 — request fix now
POST	/api/g365/commands/upload-interval	{ imei, seconds }	0x97
POST	/api/g365/commands/status-interval	{ imei, minutes }	0x13 status upload interval
POST	/api/g365/commands/heartbeat-interval	{ imei, seconds }	0x13 heartbeat
POST	/api/g365/commands/prohibit-lbs	{ imei, enabled: true false }	0x33
POST	/api/g365/commands/power	{ imei, operation: "restart" "shutdown" }	0x48
POST	/api/g365/commands/find	{ imei, start: true false }	0x49 beep
POST	/api/g365/commands/overspeed	{ imei, kmh }	0x86
POST	/api/g365/commands/phone	{ imei, role, number }	0x40–0x43 (sos, dad, mom, monitor)
POST	/api/g365/commands/expiry-date	{ imei, yyyyymmdd }	0x30
POST	/api/g365/commands/raw	{ imei, hex }	Any frame starting with 7878

Success (200):

```
{
  "ok": true,
  "imei": "123456789012345",
  "command": "0x66 redirect → 116.203.209.68:5003",
  "hex": "7878...0D0A"
}
```

Offline (503):

```
{
  "error": "device_offline",
  "imei": "...",
  "hint": "365GPS device must have an active TCP session on GPS365_TCP_PORT"
}
```

365GPS positions are stored in the **same SQLite DB** and appear under `/api/app/position` and `/api/app/history`.

Environment variables

Variable	Default	Purpose
HTTP_PORT	5002	HTTP + React static files
TCP_PORT	5001	Xexun TCP listener
GPS365_TCP_PORT	5003	365GPS TCP listener
GPS365_TCP_ENABLED	1	Set 0 to disable 365GPS listener
PERSIST_TO_SQLITE	1	Must stay 1 in production
SQLITE_PATH	data/petpal.sqlite	Database file (absolute path recommended in PM2)
WEB_BUILD_DIR	../../petpal/build	React build served as static files
HTTP_CORS_ORIGIN	*	Comma-separated origins or *
SEED_DEVICE_IMEI	—	Optional demo device
SEED_DEVICE_LAT / SEED_DEVICE_LNG	—	With seed IMEI

Deployment

```
cd ~/PetPal && git pull
cd petpal && npm ci && npm run build
cd ../tracker-tcp-server && npm ci && pm2 restart tracker
```

Verify:

```
curl -s http://YOUR_HOST:5002/api/g365 | jq .
curl -s "http://YOUR_HOST:5002/api/app/position?deviceId=YOUR_IMEI" | jq .
pm2 logs tracker --lines 20
```

Firewall: 5001/tcp, 5002/tcp, 5003/tcp (if using 365GPS).

Postman

Import:

- postman/PetPal-Tracker-API.postman_collection.json
- postman/PetPal-Tracker-API.postman_environment.json

Set collection variables `host`, `imei`, and run **Discovery** → **GET /api/g365**.

PDF export: run `npm run docs:pdf` in `tracker-tcp-server` to rebuild `API_REFERENCE.pdf` from this file.

Related docs

- `../README.md` — PM2, backups, ports
- `G365_PROTOCOL.md` — 365GPS wire format
- `../docs/TRACKING_SETUP.md` — Frontend env, HTTPS, Wi-Fi UI